

In []:

```
#1.setup
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import precision_score
import numpy as np

#2.Load and preprocess data
(train_images, train_labels), (test_images, test_labels) = keras.datasets.mnist.load_data()

# Reshape for CNN input (add channel dimension for grayscale images)
train_images = train_images.reshape((train_images.shape[0], 28, 28, 1))
test_images = test_images.reshape((test_images.shape[0], 28, 28, 1))

# Normalize
train_images = train_images / 255.0
test_images = test_images / 255.0

#3.build different CNN architectures
#.....3.1 Model 1: Simple CNN
def build_simple_cnn():
    model = keras.Sequential([
        layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        layers.MaxPooling2D((2,2)),
        layers.Conv2D(64, (3,3), activation='relu'),
        layers.MaxPooling2D((2,2)),
        layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model

# .....3.2 .Model 2: Deeper CNN
def build_deep_cnn():
    model = keras.Sequential([
        layers.Conv2D(32, (3,3), activation='relu', padding='same', input_shape=(28,28,1)),
        layers.Conv2D(32, (3,3), activation='relu'),
        layers.MaxPooling2D(),
        layers.Dropout(0.25),

        layers.Conv2D(64, (3,3), activation='relu', padding='same'),
        layers.Conv2D(64, (3,3), activation='relu'),
        layers.MaxPooling2D(),
        layers.Dropout(0.25),

        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(10, activation='softmax')
    ])
    return model

#.....3.3Model 3: Using Pretrained Architecture (Transfer Learning)
def build_transfer_model():
    # Define an input layer for the model, matching the MNIST image dimensions
    input_tensor = keras.Input(shape=(28, 28, 1))

    # Resize the input image to 32x32 and convert to 3 channels for MobileNetV2
    x = layers.Lambda(lambda image: tf.image.resize(image, (32, 32)))(input_tensor)
    x = layers.Lambda(lambda image: tf.image.grayscale_to_rgb(image))(x)

    # Load MobileNetV2 with the required input shape (32x32x3)
    base_model = keras.applications.MobileNetV2(
        input_shape=(32,32,3),
        include_top=False,
        weights=None # Keep weights=None to train from scratch
    )

    # Connect the preprocessed input to the base model
    x = base_model(x)

    # Add the rest of the layers for classification
    x = layers.GlobalAveragePooling2D()(x)
    output_tensor = layers.Dense(10, activation='softmax')(x)

    model = keras.Model(inputs=input_tensor, outputs=output_tensor)
    return model

#.....3.4 Compile & Train
def compile_and_train(model):
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    history = model.fit(
```

```

        train_images, train_labels,
        epochs=10,
        validation_split=0.2,
        batch_size=64
    )
    return history

#...3.5 Evaluate Performance (Accuracy + Precision)
def evaluate_model(model):
    # Accuracy
    test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=0)

    # Predictions
    y_pred = model.predict(test_images)
    y_pred_classes = np.argmax(y_pred, axis=1)

    # Precision (macro for multi-class). MNIST test_labels are 1D.
    precision = precision_score(test_labels, y_pred_classes, average='macro')

    return test_acc, precision

#.....3.6 RUN EXPERIMENTS
models = {
    "Simple CNN": build_simple_cnn(),
    "Deep CNN": build_deep_cnn(),
    "Transfer Model": build_transfer_model()
}

results = {}

for name, model in models.items():
    print(f"Training {name}...")
    compile_and_train(model)
    acc, prec = evaluate_model(model)
    results[name] = {"Accuracy": acc, "Precision": prec}

print(results)

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434  **0s** 0us/step

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`
`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the
model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Training Simple CNN...

Epoch 1/10
750/750  **49s** 60ms/step - accuracy: 0.9390 - loss: 0.2024 - val_accuracy: 0.9810 - val_loss: 0.0666
Epoch 2/10
750/750  **76s** 52ms/step - accuracy: 0.9822 - loss: 0.0581 - val_accuracy: 0.9839 - val_loss: 0.0520
Epoch 3/10
750/750  **38s** 49ms/step - accuracy: 0.9871 - loss: 0.0409 - val_accuracy: 0.9864 - val_loss: 0.0482
Epoch 4/10
716/750  **1s** 50ms/step - accuracy: 0.9903 - loss: 0.0289